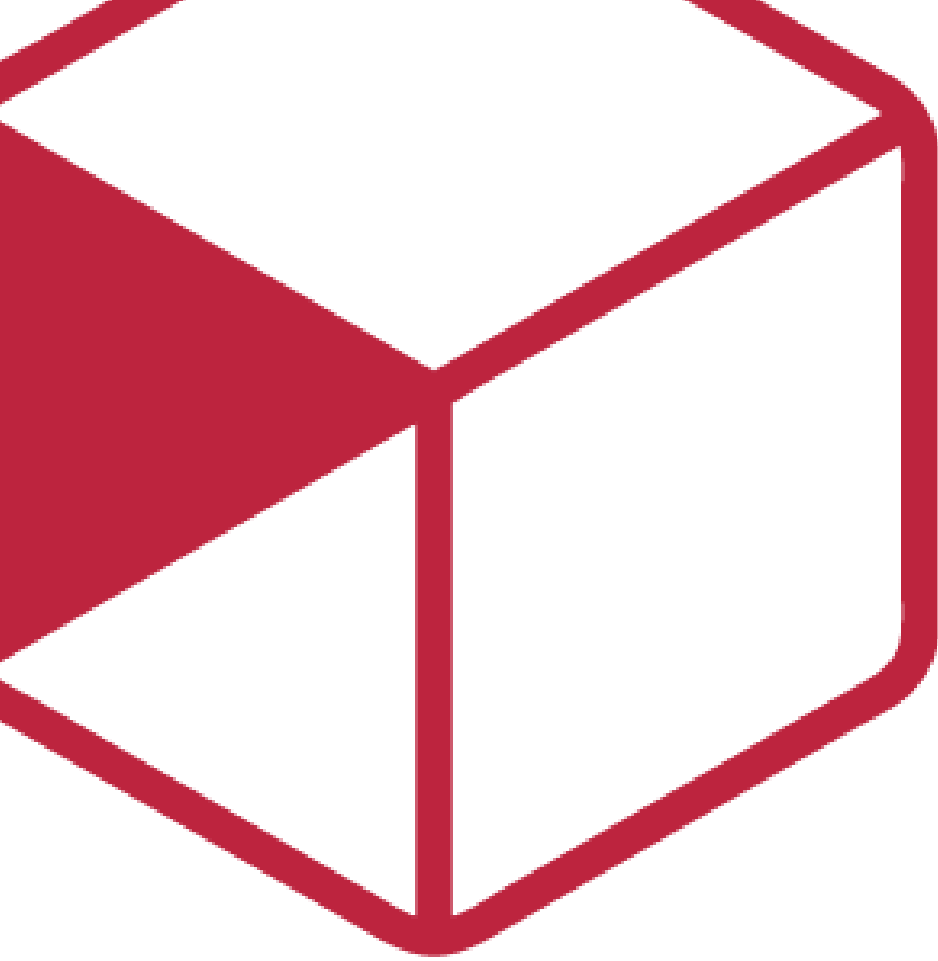




Diseño de la solución

Requisito R16A-RE-FU-005

FORMATO	Arquitectura
PROYECTO	R16 - Adquisiciones
REFERENCIA	AUI- FOR-01
VERSIÓN	1.5
FECHA	18/06/2026
AUTOR	Cristóbal Sebastián García Coss
REVISOR	Juan David García Cruz

Contenido

Requisito R16A-RE-FU-005	3
Contenido	3
Proposito del documento	3
Alcance	3
Incluye:	3
No incluye:	4
Objetivo tecnico	4
Componentes involucrados	4
Flujo 1 — Consulta de catalogo con filtro de region	5
Flujo 2 — Guardado de datos de facturacion del cliente	5
Flujo 3 — Consulta de detalle de facturacion del cliente (DatosFacturacionClienteObj)	6
Criterios de aceptacion del requisito	6
Reglas tecnicas aplicadas	8
Cambios en Base de Datos	9
Paso 1 — ALTER TABLE + UPDATE masivo (prerequisito)	9
Paso 2 — INSERT registros Peru	10
Paso 3 — ALTER banderas tributarias (CONDICIONAL — solo tras confirmar P5/P6)	10
Paso 4 — ALTER VIEW (CONDICIONAL — solo tras Paso 3)	10
Cambios en Controllers C#	10
catMetodoDePagoCFDIController.cs — Estado final	10
catUsoCFDIController.cs — Estado final	11
catMedioDePagoController.cs — Estado final	12
Actualizacion EDMX	13
Diagrama de secuencia — Consulta de catalogo con filtro de region	13
Impacto en codigo existente	14
Archivos modificados	14
Archivos NO modificados (confirmado)	14
Impacto en modelos por actualizacion EDMX	15
Pruebas funcionales (Criterios de Aceptacion en DEV)	16
Pruebas tecnicas	16
Unitarias	16
Integracion	16
Casos criticos	17
Acciones requeridas antes de implementar	19
Control de versiones	19



Proposito del documento

Definir el diseño de la solución técnica para el requisito **R16A-RE-FU-005 (Configuración de Cobros y Facturación del Cliente)**, describiendo los cambios en base de datos, la capa de controllers WebApi, el modelo EDMX y las validaciones necesarias para habilitar catálogos de cobros diferenciados por región (Mexico / Peru).

Nota: Este documento cubre exclusivamente el diseño **Backend** (.NET Framework 4.8, EF6 Database-First, WebApi 2). El diseño Frontend (Angular, NgRx) se documenta por separado.

Alcance

Incluye:

- ALTER TABLE en 3 catálogos (`catMetodoDePagoCFDI`, `catUsoCFDI`, `catMedioDePago`) para agregar `IdRegion`
- INSERT de registros Peru en `catMetodoDePagoCFDI` (Contado/Credito) y `catUsoCFDI` (Factura/Boleta/Recibo)
- ALTER TABLE en `DatosFacturacionCliente` para banderas tributarias PE (condicional)
- Modificación de 3 controllers para heredar `BaseApiController` y aplicar filtro de región
- Actualización del EDMX para reflejar cambios estructurales
- Actualización de vista `vDatosFacturacionCliente` (condicional)

No incluye:

- Lógica de cálculo de retenciones/detracciones en módulos de facturación
- Habilitación de facturación electrónica SUNAT (brechas 1–5 fuera de alcance)
- Cambios en `DatosFacturacionClienteBO`, `DatosFacturacionClienteFactory` ni `DatosFacturacionClienteVDBO`
- Renderizado condicional de campos en Frontend
- Cambios en módulos consumidores (Factura por Adelantado, Validar Cobro)

Objetivo tecnico

Agregar la dimensión de región (`IdRegion`) a los catálogos de cobros del sistema para que los endpoints devuelvan únicamente las opciones correspondientes a la región del usuario autenticado. Insertar los registros de Peru en los catálogos reutilizados. Opcionalmente, agregar campos de banderas tributarias peruanas a la entidad de facturación del cliente.

Componentes involucrados

Aplicativo	Componente	Responsabilidad	Ubicación
------------	------------	-----------------	-----------



ProquifaDotNet (BD)	catMetodoDePagoCFDI	Almacenar opciones MX (PUE/PPD) y PE (Contado/Credito) diferenciadas por region	SQL Server
ProquifaDotNet (BD)	catUsoCFDI	Almacenar opciones MX (G01/G03/S01) y PE (01/03/08) diferenciadas por region	SQL Server
ProquifaDotNet (BD)	catMedioDePago	Almacenar opciones de Forma de Pago (solo MX) con region asignada	SQL Server
ProquifaDotNet (BD)	DatosFacturacionCliente	Almacenar banderas tributarias PE (condicional)	SQL Server
ProquifaDotNet (BD)	vDatosFacturacionCliente	Exponer datos de facturacion incluyendo campos PE nuevos	SQL Server
WebApi.Catalogos	catMetodoDePagoCFDIController	Servir catalogo filtrado por region del usuario	Controllers\Catalogos\
WebApi.Catalogos	catUsoCFDIController	Servir catalogo filtrado por region del usuario	Controllers\Catalogos\
WebApi.Catalogos	catMedioDePagoController	Servir catalogo filtrado por region del usuario	Controllers\Catalogos\
Core.WebApi	BaseApiController	Proveer ObtenerUsuarioAutenticado() y AsegurarFiltroRegion()	_WebApi\Core.WebApi\ControllerOperations\
Core.Pqf	ProquifaDotNetModel.edmx	Modelo EF6 Database-First reflejar columnas nuevas	Core.Pqf\ProquifaDotNetContext\

Flujo 1 — Consulta de catalogo con filtro de region

1. Frontend solicita POST /catMetodoDePagoCFDI con QueryInfo (filtros, paginacion).



2. El controller recibe el request en `QueryResult([FromBody] QueryInfo info)`.
3. El controller llama `ObtenerUsuarioAutenticado()` que extrae el token del header `Authorization`, lo valida contra `IdentityServer4` y retorna el objeto `Usuario` con su `IdRegion`.
4. El controller llama `AsegurarFiltroRegion(info, user.IdRegion)`:
 - Si `info.Filters` no tiene filtro `IdRegion`, lo agrega con el valor de `user.IdRegion`.
 - Si ya tiene filtro `IdRegion` pero vacío, lo completa con `user.IdRegion`.
 - Si `user.IdRegion` es `Guid.Empty`, lanza `ArgumentException`.
5. El controller instancia `TablaGenericaBO<catMetodoDePagoCFDI>()` y llama `QueryResult(info)`.
6. EF6 ejecuta la consulta con el filtro `WHERE IdRegion = @userRegion AND Activo = 1`.
7. Retorna `QueryResult<catMetodoDePagoCFDI>` con solo registros de la region del usuario:
 - Usuario MEX → PUE, PPD
 - Usuario PER → Contado, Credito

Aplica idénticamente a `/catUsoCFDI` y `/catMedioDePago`.

Flujo 2 — Guardado de datos de facturación del cliente

1. Frontend envía `PUT /DatosFacturacionCliente` con la entidad modificada (incluye `IdCatMetodoDePagoCFDI` e `IdCatUsoCFDI` seleccionados).
2. `DatosFacturacionClienteController.GuardarOActualizar()` delega a `DatosFacturacionClienteVDBO`.
3. `DatosFacturacionClienteVDBO` llama a `DatosFacturacionClienteBO._GuardarOActualizar()`:
 - Asigna defaults (moneda, empresa por region, tipo de cambio).
 - Ejecuta upsert por `IdCliente` (soft upsert via `_AddOrUpdateUniqueElement`).
 - Sincroniza datos STP y legacy client.
4. Los FKs `IdCatMetodoDePagoCFDI` e `IdCatUsoCFDI` apuntan a registros que ya están filtrados por region en consulta. **No se requiere validación adicional de region en guardado** porque el catálogo filtra por region al obtener las opciones; el usuario solo puede seleccionar opciones de su region.

Sin cambios en este flujo para R16. Los FKs existentes funcionan transparentemente con los nuevos registros PE.

Flujo 3 — Consulta de detalle de facturación del cliente (DatosFacturacionClienteObj)

1. Frontend solicita `GET /DatosFacturacionClienteDetalle?idCliente={guid}`.
2. `DatosFacturacionClienteDetalleController.Obtener()` delega a `DatosFacturacionClienteDetalleBO`.



3. `DatosFacturacionClienteBO.Extensions.ObtenerPorCliente()` busca el registro o lo crea via `DatosFacturacionClienteFactory`.

4. `DatosFacturacionClienteObj` (modelo enriquecido) carga eager:

- `catUsoCFDI` — mostrara G01/G03 para MX o Factura/Boleta para PE segun la FK almacenada.

- `catMetodoDePagoCFDI` — mostrara PUE/PPD para MX o Contado/Credito para PE.

- `catMedioDePago` via `ConfiguracionPagos` — solo MX; PE no tendra registro.

5. Retorna al frontend el objeto completo.

Sin cambios en este flujo. La carga eager funciona transparentemente con las FKs existentes.

Criterios de aceptacion del requisito

CA	Descripcion	Estado	Justificacion
A1	Campos renderizados segun Region del cliente	Cubierto	Controllers filtran por <code>IdRegion</code> via <code>AsegurarFiltroRegion</code> . Frontend renderiza condicionalmente.
B1	Selector Forma de Pago con catalogo MX	Cubierto	<code>catMedioDePago</code> con <code>IdRegion=MEX</code> retorna: Cheque, Deposito bancario, Efectivo, Tarjeta, Transferencia, Otros, Na, —ninguno—.
B2	Selector Uso de CFDI con catalogo MX	Cubierto	<code>catUsoCFDI</code> con <code>IdRegion=MEX</code> retorna: G01, G02, G03, S01, P01, N/A.
B3	Selector Metodo de Pago con catalogo SAT	Cubierto	<code>catMetodoDePagoCFDI</code> con <code>IdRegion=MEX</code> retorna: PUE, PPD.
C1	Selector Condicion de Pago con catalogo SUNAT	Cubierto	<code>catMetodoDePagoCFDI</code> con <code>IdRegion=PER</code> retorna: Contado, Credito (INSERT en T2).
C2	Selector Tipo de Comprobante con catalogo SUNAT	Cubierto	<code>catUsoCFDI</code> con <code>IdRegion=PER</code> retorna: 01 Factura, 03 Boleta, 08 Recibo (INSERT en T2).
C3	Metodo de Pago y Forma de Pago no renderizados para Peru	Cubierto	<code>catMedioDePago</code> sin registros PER → endpoint retorna vacio. <code>catMetodoDePagoCFDI</code> con <code>IdRegion=PER</code> solo retorna CONT/CRED. Frontend oculta campos no aplicables.
D1	Bandera Agente de Retencion IGV	Condicional	<code>DatosFacturacionCliente.AgenteRetencionIGV</code> bit — solo si P5 se confirma.
D2	Bandera Sujeto a Detraccion	Condicional	<code>DatosFacturacionCliente.SujetoDetraccion + TasaDetraccion</code> — solo si P6 se confirma.

E1	Obligatoriedad guardar (MX)	al Frontend	Backend: FKs son NULLABLE en BD. Obligatoriedad se valida en frontend antes de enviar.
E2	Obligatoriedad guardar (PE)	al Frontend	Idem.
E3	Edicion sin restriccion de rol	Cubierto	Sin control de rol en backend — acceso por cartera del cliente. Sin cambios.
E4	Persistencia como configuracion default	Cubierto	DatosFacturacionCliente y ConfiguracionPagos ya funcionan como defaults consumidos por modulos. Sin cambios.

Reglas tecnicas aplicadas

Regla	Descripcion
RT-01	Los 3 controllers DEBEN heredar <code>BaseApiController</code> y NO <code>ApiController</code> . Se elimina <code>IControllerCatalogoBO<T></code> de la declaracion de clase.
RT-02	El metodo <code>QueryResult</code> de cada controller DEBE llamar <code>ObtenerUsuarioAutenticado()</code> y <code>AsegurarFiltroRegion(info, user.IdRegion)</code> ANTES de instanciar el BO.
RT-03	Los metodos <code>Obtener</code> , <code>GuardarOActualizar</code> y <code>Desactivar</code> NO se modifican — no requieren filtro de region porque operan por PK.
RT-04	El using <code>Core.Pqf.WebApi.ControllerOperations</code> DEBE agregarse. Los usings <code>Core.Pqf.Interfaces._ABC</code> se eliminan (ya no se implementa <code>IControllerCatalogoBO</code>).
RT-05	Los scripts SQL DEBEN ejecutarse en orden: Paso 1 (ALTER+UPDATE) → Paso 2 (INSERT PE) → Paso 3 (ALTER banderas, condicional).
RT-06	El EDMX DEBE actualizarse via "Update Model from Database" despues de cada batch de ALTERs. No editar el EDMX manualmente.
RT-07	<code>IdRegion</code> en catalogos es NULLABLE con FK a <code>Region</code> . Se asigna <code>NOT NULL</code> logico via UPDATE masivo, pero la columna permanece nullable para compatibilidad con el flujo de creacion de EF6.

Cambios en Base de Datos

Paso 1 — ALTER TABLE + UPDATE masivo (prerequisito)

```
-- 1a. catMetodoDePagoCFDI
ALTER TABLE dbo.catMetodoDePagoCFDI
    ADD IdRegion uniqueidentifier NULL
    CONSTRAINT FK_catMetodoDePagoCFDI_Region
    FOREIGN KEY REFERENCES dbo.Region(IdRegion);

UPDATE dbo.catMetodoDePagoCFDI
```



```

SET      IdRegion = (SELECT IdRegion FROM dbo.Region WHERE ClaveISO = 'MEX')
WHERE    IdRegion IS NULL;

-- 1b. catUsoCFDI
ALTER TABLE dbo.catUsoCFDI
    ADD IdRegion uniqueidentifier NULL
    CONSTRAINT FK_catUsoCFDI_Region
        FOREIGN KEY REFERENCES dbo.Region(IdRegion);

UPDATE dbo.catUsoCFDI
SET      IdRegion = (SELECT IdRegion FROM dbo.Region WHERE ClaveISO = 'MEX')
WHERE    IdRegion IS NULL;

-- 1c. catMedioDePago
ALTER TABLE dbo.catMedioDePago
    ADD IdRegion uniqueidentifier NULL
    CONSTRAINT FK_catMedioDePago_Region
        FOREIGN KEY REFERENCES dbo.Region(IdRegion);

UPDATE dbo.catMedioDePago
SET      IdRegion = (SELECT IdRegion FROM dbo.Region WHERE ClaveISO = 'MEX')
WHERE    IdRegion IS NULL;

-- Verificacion
SELECT 'catMetodoDePagoCFDI' AS Tabla, COUNT(*) AS SinRegion FROM dbo.catMetodoDePagoCFDI
WHERE IdRegion IS NULL
UNION ALL
SELECT 'catUsoCFDI', COUNT(*) FROM dbo.catUsoCFDI WHERE IdRegion IS NULL
UNION ALL
SELECT 'catMedioDePago', COUNT(*) FROM dbo.catMedioDePago WHERE IdRegion IS NULL;
-- Los 3 deben retornar 0

```

Paso 2 — INSERT registros Peru

```

DECLARE @IdPeru uniqueidentifier = (SELECT IdRegion FROM dbo.Region WHERE ClaveISO = 'PER');

-- Condicion de Pago SUNAT
INSERT INTO dbo.catMetodoDePagoCFDI
    (IdCatMetodoDePagoCFDI, MetodoDePagoCFDI, ClaveMetodoDePagoCFDI, Clave, Activo, IdRegion)
VALUES
    (NEWID(), N'Contado', 'CONT', 'CONT', 1, @IdPeru),
    (NEWID(), N'Crédito', 'CRED', 'CRED', 1, @IdPeru);

-- Tipo de Comprobante SUNAT
INSERT INTO dbo.catUsoCFDI
    (IdCatUsoCFDI, ClaveUso, Uso, Clave, Activo, IdRegion)
VALUES
    (NEWID(), '01', N'Factura electrónica', '01', 1, @IdPeru),
    (NEWID(), '03', N'Boleta de venta electrónica', '03', 1, @IdPeru),
    (NEWID(), '08', N'Recibo por Honorarios electrónico', '08', 1, @IdPeru);

-- Verificacion
SELECT ClaveMetodoDePagoCFDI, MetodoDePagoCFDI, IdRegion FROM dbo.catMetodoDePagoCFDI WHERE
IdRegion = @IdPeru;
SELECT ClaveUso, Uso, IdRegion FROM dbo.catUsoCFDI WHERE IdRegion = @IdPeru;

```

Paso 3 — ALTER banderas tributarias (CONDICIONAL — solo tras confirmar P5/P6)




```
ALTER TABLE dbo.DatosFacturacionCliente
    ADD AgenteRetencionIGV bit NOT NULL CONSTRAINT DF_DFC_AgenteRetencionIGV
DEFAULT 0,
    SujetoDetraccion bit NOT NULL CONSTRAINT DF_DFC_SujetoDetraccion
DEFAULT 0,
    TasaDetraccion decimal(5,2) NULL;
```

Paso 4 — ALTER VIEW (CONDICIONAL — solo tras Paso 3)

```
-- Ejecutar SELECT OBJECT_DEFINITION(OBJECT_ID('dbo.vDatosFacturacionCliente'))
-- para obtener definicion actual. Agregar las 3 columnas al SELECT:
-- dfc.AgenteRetencionIGV, dfc.SujetoDetraccion, dfc.TasaDetraccion
```

Cambios en Controllers C#

catMetodoDePagoCFDIController.cs — Estado final

```
#region

using System;
using System.Web.Http;
using Core.CrudTools.Optimization;
using Core.Pqf.BusinessBasicTools._Misc.Crud;
using Core.Pqf.ProquifaDotNetContext;
using Core.Pqf.WebApi.ControllerOperations;

#endregion

namespace WebApi.Controllers.Catalogos
{
    public class catMetodoDePagoCFDIController : BaseApiController
    {
        [HttpGet]
        [Route("catMetodoDePagoCFDI")]
        public catMetodoDePagoCFDI Obtener(Guid idcatMetodoDePagoCFDI)
        {
            var modelBO = new TablaGenericaBO<catMetodoDePagoCFDI>();
            return modelBO.Obtener(idcatMetodoDePagoCFDI);
        }

        [HttpPut]
        [Route("catMetodoDePagoCFDI")]
        public Guid GuardarOActualizar([FromBody] catMetodoDePagoCFDI catMetodoDePagoCFDI)
        {
            var modelBO = new TablaGenericaBO<catMetodoDePagoCFDI>();
            return modelBO.GuardarOActualizar(catMetodoDePagoCFDI);
        }

        [HttpPost]
        [Route("catMetodoDePagoCFDI")]
        public QueryResult<catMetodoDePagoCFDI> QueryResult([FromBody] QueryInfo info)
        {
            var user = ObtenerUsuarioAutenticado();
            AsegurarFiltroRegion(info, user.IdRegion);
            var modelBO = new TablaGenericaBO<catMetodoDePagoCFDI>();
            return modelBO.QueryResult(info);
        }
    }
}
```



```
[HttpDelete]
[Route("catMetodoDePagoCFDI")]
public Guid Desactivar(Guid idcatMetodoDePagoCFDI)
{
    var modelBO = new TablaGenericaBO<catMetodoDePagoCFDI>();
    return modelBO.Desactivar(idcatMetodoDePagoCFDI);
}
}
```

catUsoCFDIController.cs — Estado final

```
#region

using System;
using System.Web.Http;
using Core.CrudTools.Optimization;
using Core.Pqf.BusinessBasicTools._Misc.Crud;
using Core.Pqf.ProquifaDotNetContext;
using Core.Pqf.WebApi.ControllerOperations;

#endregion

namespace WebApi.Controllers.Catalogos
{
    public class catUsoCFDIController : BaseApiController
    {
        [HttpGet]
        [Route("catUsoCFDI")]
        public catUsoCFDI Obtener(Guid idcatUsoCFDI)
        {
            var modelBO = new TablaGenericaBO<catUsoCFDI>();
            return modelBO.Obtener(idcatUsoCFDI);
        }

        [HttpPut]
        [Route("catUsoCFDI")]
        public Guid GuardarOActualizar([FromBody] catUsoCFDI catUsoCFDI)
        {
            var modelBO = new TablaGenericaBO<catUsoCFDI>();
            return modelBO.GuardarOActualizar(catUsoCFDI);
        }

        [HttpPost]
        [Route("catUsoCFDI")]
        public QueryResult<catUsoCFDI> QueryResult([FromBody] QueryInfo info)
        {
            var user = ObtenerUsuarioAutenticado();
            AsegurarFiltroRegion(info, user.IdRegion);
            var modelBO = new TablaGenericaBO<catUsoCFDI>();
            return modelBO.QueryResult(info);
        }

        [HttpDelete]
        [Route("catUsoCFDI")]
        public Guid Desactivar(Guid idcatUsoCFDI)
        {
            var modelBO = new TablaGenericaBO<catUsoCFDI>();
            return modelBO.Desactivar(idcatUsoCFDI);
        }
    }
}
```



```
}  
}
```

catMedioDePagoController.cs — Estado final

```
#region  
  
using System;  
using System.Web.Http;  
using Core.CrudTools.Optimization;  
using Core.Pqf.BusinessBasicTools._Misc.Crud;  
using Core.Pqf.ProquifaDotNetContext;  
using Core.Pqf.WebApi.ControllerOperations;  
  
#endregion  
  
namespace WebApi.Controllers.Catalogos  
{  
    public class catMedioDePagoController : BaseApiController  
    {  
        [HttpGet]  
        [Route("catMedioDePago")]  
        public catMedioDePago Obtener(Guid idcatMedioDePago)  
        {  
            var modelBO = new TablaGenericaBO<catMedioDePago>();  
            return modelBO.Obtener(idcatMedioDePago);  
        }  
  
        [HttpPut]  
        [Route("catMedioDePago")]  
        public Guid GuardarOActualizar([FromBody] catMedioDePago catMedioDePago)  
        {  
            var modelBO = new TablaGenericaBO<catMedioDePago>();  
            return modelBO.GuardarOActualizar(catMedioDePago);  
        }  
  
        [HttpPost]  
        [Route("catMedioDePago")]  
        public QueryResult<catMedioDePago> QueryResult([FromBody] QueryInfo info)  
        {  
            var user = ObtenerUsuarioAutenticado();  
            AsegurarFiltroRegion(info, user.IdRegion);  
            var modelBO = new TablaGenericaBO<catMedioDePago>();  
            return modelBO.QueryResult(info);  
        }  
  
        [HttpDelete]  
        [Route("catMedioDePago")]  
        public Guid Desactivar(Guid idcatMedioDePago)  
        {  
            var modelBO = new TablaGenericaBO<catMedioDePago>();  
            return modelBO.Desactivar(idcatMedioDePago);  
        }  
    }  
}
```

Actualizacion EDMX



1. Abrir `Core.Pqf\ProquiifaDotNetContext\ProquiifaDotNetModel.edmx` en Visual Studio.
2. Click derecho → "Update Model from Database".
3. Seleccionar las tablas modificadas: `catMetodoDePagoCFDI`, `catUsoCFDI`, `catMedioDePago`, y si aplica `DatosFacturacionCliente`.
4. Actualizar. Las clases `.tt` se regeneran automaticamente.
5. Verificar que las entidades generadas contienen la propiedad `IdRegion` (tipo `Nullable<Guid>`).
6. Compilar solucion. Corregir errores si los hay.
7. Commitear EDMX + `.tt` + `.cs` generados.

Diagrama de secuencia — Consulta de catalogo con filtro de region

Frontend EF6/BD	Controller	BaseApiController	TablaGenericaBO
 -- POST /catUsoCFDI --->			
 (QueryInfo)			
		 -- ObtenerUsuarioAutenticado() -->	
		 <-- Usuario {IdRegion} -----	
		 -- AsegurarFiltroRegion(info, IdRegion) -->	
		 [agrega FilterTuple("IdRegion", val)]	
		 <-----	
		 -- new TablaGenericaBO<catUsoCFDI>() ----->	
		 -- QueryResult(info) ----->	
WHERE			 -- SELECT
IdRegion=@val -->			
			 <-- rows

		 <-- QueryResult<catUsoCFDI> -----	
	 <-- JSON response -----		

Impacto en código existente

Archivos modificados

#	Archivo	Cambio exacto
1	catMetodoDePagoCFDIController.cs	Linea 14: ApiController, IControllerCatalogoBO<catMetodoDePagoCFDI> → BaseApiController. Lineas 50-53: agregar + ObtenerUsuarioAutenticado() AsegurarFiltroRegion(). Using: reemplazar por Core.Pqf.Interfaces._ABC Core.Pqf.WebApi.ControllerOperations.
2	catUsoCFDIController.cs	Mismo patron que #1.
3	catMedioDePagoController.cs	Mismo patron que #1.

Archivos NO modificados (confirmado)

Archivo	Razon
DatosFacturacionClienteBO.cs	FKs existentes funcionan transparentemente con nuevos registros PE
DatosFacturacionClienteBO.Extensions.cs	ObtenerPorCliente() no depende de region
DatosFacturacionClienteFactory.cs	Factory crea defaults; la seleccion de catalogo de cobros se resuelve en frontend
DatosFacturacionClienteObj.cs	Carga eager via FK — funciona transparentemente
DatosFacturacionClienteController.cs	Save delega a BO. Sin logica de catalogo
DatosFacturacionClienteDetalleController.cs	Read-only. Sin logica de filtro
ConfiguracionPagosBO.cs	IdCatMedioDePago FK ya existe. Sin cambios
vDatosFacturacionClienteBO.cs	Sin cambios (salvo que se ejecute Paso 3/4 BD)

Impacto en modelos por actualización EDMX

Tras "Update Model from Database":

Entidad	Propiedad nueva	Tipo C# generado
catMetodoDePagoCFDI	IdRegion	Nullable<Guid>
catUsoCFDI	IdRegion	Nullable<Guid>
catMedioDePago	IdRegion	Nullable<Guid>
DatosFacturacionCliente (condicional)	AgenteRetencionIGV	bool
DatosFacturacionCliente (condicional)	SujetoDetraccion	bool



DatosFacturacionCliente TasaDetraccion Nullable<decimal>
(condicional)

Propiedad de navegacion nueva en cada catalogo: Region (referencia a entidad Region).

Escenario	Comportamiento esperado
Usuario sin IdRegion asignada (Guid.Empty)	AsegurarFiltroRegion lanza InvalidOperationException con mensaje "El usuario no tiene una región asignada." → HTTP 500 capturado por BusinessExceptionHandler.
Token de autorizacion invalido o expirado	ObtenerUsuarioAutenticado() lanza excepcion → HTTP 401 via AuthorizeProquifaDotNet filter global.
QueryInfo nulo en request	AsegurarFiltroRegion lanza ArgumentNullException → HTTP 500.
INSERT de registro PE duplicado (re-ejecucion de script)	NEWID() genera PK unico → no hay conflicto. Para idempotencia, agregar IF NOT EXISTS al script.
FK rota por IdRegion apuntando a region inexistente	La FK REFERENCES dbo.Region(IdRegion) previene la insercion → error SQL.
Cliente PE intenta guardar un IdCatUsoCFDI de MX	No deberia ocurrir: el selector del frontend solo muestra opciones filtradas por region. Si ocurre por manipulacion directa del request, la FK acepta cualquier ID valido — no hay validacion cruzada cliente-region-catalogo en backend. Riesgo aceptado: la probabilidad es minima dado que el frontend filtra correctamente.

Pruebas funcionales (Criterios de Aceptacion en DEV)

- Verificar que un usuario MEX consultando `POST /catMetodoDePagoCFDI` recibe solo PUE y PPD.
- Verificar que un usuario PER consultando `POST /catMetodoDePagoCFDI` recibe solo Contado y Credito.
- Verificar que un usuario MEX consultando `POST /catUsoCFDI` recibe G01, G02, G03, S01, P01, N/A.
- Verificar que un usuario PER consultando `POST /catUsoCFDI` recibe 01, 03, 08.
- Verificar que un usuario PER consultando `POST /catMedioDePago` recibe un resultado vacio (0 registros).
- Verificar que un usuario MEX consultando `POST /catMedioDePago` recibe los registros existentes (Cheque, Deposito, Efectivo, etc.).
- Verificar que `GET /DatosFacturacionClienteDetalle` de un cliente PER retorna el `catUsoCFDI` correcto (01/03/08) y el `catMetodoDePagoCFDI` correcto (CONT/CRED).

Pruebas tecnicas



Unitarias

- Verificar que `AsegurarFiltroRegion` agrega filtro `IdRegion` cuando no existe en `info.Filters`.
- Verificar que `AsegurarFiltroRegion` completa el valor cuando el filtro existe pero esta vacío.
- Verificar que `AsegurarFiltroRegion` lanza excepcion cuando `userRegion` es `Guid.Empty`.
- Verificar que `AsegurarFiltroRegion` lanza excepcion cuando `info` es `null`.

Integracion

- Con BD local, ejecutar los scripts Paso 1 y Paso 2, luego consultar `catMetodoDePagoCFDI` filtrando por `IdRegion PER` → debe retornar 2 registros.
- Con BD local, consultar `catUsoCFDI` filtrando por `IdRegion PER` → debe retornar 3 registros.
- Con BD local, verificar que registros MEX existentes tienen `IdRegion` asignado y no fueron alterados.

Casos criticos

- **Usuario sin region:** Simular un usuario con `IdRegion = Guid.Empty` y verificar que el endpoint retorna HTTP 500 con mensaje descriptivo, no un resultado sin filtrar.
- **Concurrencia de scripts:** Si Paso 2 se ejecuta antes de Paso 1 (`IdRegion` no existe), el INSERT falla porque la columna no existe. Los scripts tienen dependencia estricta y deben ejecutarse en orden.
- **Rollback de EDMX:** Si se actualiza el EDMX antes de ejecutar los scripts en BD de desarrollo, la compilacion falla porque las propiedades no existen en la tabla. Sincronizar BD primero, EDMX despues.

#	Pendiente	Impacto en este diseño	Responsable	Estado
P1	Denominación final "Condición de Pago" para Perú (¿Contado/Crédito u otra nomenclatura?)	Afecta campo <code>MetodoDePagoCFDI</code> del INSERT en Paso 2. Si cambia la denominación, ajustar el valor insertado.	Funcional / Cliente	Pendiente
P2	Mapeo legacy de catálogos: ¿facturas y pedidos históricos necesitan ajuste al agregar <code>IdRegion</code> ?	Si se requiere mapeo, podría necesitarse un script adicional de migración de datos post-Paso 1. Sin el mapeo, los históricos quedan con	Funcional / TechLead	Pendiente



		IdRegion=MEX (correcto para datos existentes).		
P3	Tasa de detracción: ¿se captura a nivel cliente, producto o catálogo SUNAT?	Define si <code>TasaDetraccion</code> en <code>DatosFacturacionCliente</code> (Paso 3) es la ubicación correcta o si se necesita un campo adicional en catálogo de productos.	Funcional Cliente	/ Pendiente
P4	Clave SAT para Aba, NA, —NINGUNO— y Swift en <code>catMedioDePago</code>	Sin clave SAT, el timbrado CFDI puede fallar para clientes MX con estos valores. Script de UPDATE pendiente (Tarea T6). No bloquea este diseño pero sí el timbrado.	Funcional Cliente	/ Pendiente
P5	¿PROQUIFA Perú tiene clientes designados Agentes de Retención IGV por SUNAT?	Bloquea Paso 3 (ALTER banderas). Si la respuesta es No, la columna <code>AgenteRetencionIGV</code> no se crea y se ahorra desarrollo.	Funcional Cliente	/ Bloqueante
P6	¿Productos/servicios de PROQUIFA Perú están sujetos a Detracción SPOT?	Bloquea Paso 3 (ALTER banderas). Si la respuesta es No, las columnas <code>SujetoDetraccion</code> y <code>TasaDetraccion</code> no se crean.	Funcional Cliente	/ Bloqueante
P7	¿PROQUIFA Perú (Golcaer) es Agente de Percepción IGV?	Condición del emisor, no del cliente. De aplicar, se configura a nivel sistema (Brecha 4). No afecta este diseño directamente pero sí el módulo de facturación.	Funcional Cliente	/ Pendiente
P8	Campo "Tipo de Revisión" (Digital / Física / Híbrida):	Si entra, podría requerir campo	Funcional Cliente	/ Pendiente



¿entra en alcance R16? adicional en
¿aplica a Perú? DatosFacturacionClie
nte o catálogo nuevo
con IdRegion. No
contemplado en este
diseño.

Acciones requeridas antes de implementar

1. **Resolver P5 y P6 con el cliente** — Sin respuesta, Paso 3 (ALTER banderas), Paso 4 (ALTER VIEW) y Tarea T5 no se pueden ejecutar.
2. **Confirmar P1** — Si la denominación cambia, ajustar el INSERT de Paso 2 antes de ejecutar.
3. **Validar P2 con TechLead** — Determinar si se necesita script de migración para facturas/pedidos históricos.
4. **Presentar brechas consolidadas al cliente (OBS-009)** — Listar las 5 brechas de facturación electrónica Perú para que el cliente decida alcance R16 vs release posterior.

Version	Fecha	Autor	Tipo de cambio	Descripcion del cambio	Aprobo
1.0	22/06/2026	Cristobal Garcia	Creacion	Diseno inicial de la solucion backend para R16A-RE-FU-005	

Control de versiones

Version	Fecha	Autor	Tipo de Cambio	Descripcion del cambio	Aprobo
1.0	22/06/2026	Cristobal Garcia	Creacion	Creacion del documento.	

